INTELLIGENT HEALTHCARE DIAGNOSTIC ASSISTANT

AI CAPSTONE PROJECT REPORT

Unit Code: CCS 3101

Unit Name: Artificial Intelligence

Project Title: Intelligent Healthcare Diagnostic Assistant

Repository Name: diagnosisassistant

Team Members:

Adelaide Myra C026-01-2342/2024

Beth Ngunju C026-01-0764/2024

Hannah Ng'endo C026-01-2421/2024

Angel Matoroki C026-01-2622/2024

Morara Michelle C026-01-0731/2024

Hillary Muthee C026-01-0801

Lecturer/Supervisor: Mr Kaburu

Submission Date: August 2026

EXECUTIVE SUMMARY

The Intelligent Healthcare Diagnostic Assistant is an end-to-end, multi-paradigm Artificial Intelligence platform designed to demonstrate the real-world integration of symbolic logic, probabilistic reasoning, machine learning, deep learning, fuzzy logic, and automated planning into a unified medical decision-support system.
The system receives structured patient intakes including discrete symptom sets, age, body temperature, heart rate, and blood pressure. An intelligent coordinator agent manages the diagnostic workflow by passing input states to seven specialized AI modules. Each module processes the data using a distinct AI methodology, returning disease predictions, confidence estimates, risk/severity scores, or step-by-step clinical intervention plans.
The platform integrates:

1. Module 1: Model-Based / Goal-Based Intelligent Agent Core
2. Module 2: Knowledge Base & Inference Engine (First-Order Logic)
3. Module 3: Bayesian Diagnostic Module (Log-Space Probabilistic Inference)
4. Module 4: Supervised Machine Learning Ensemble Classifier
5. Module 5: Multi-Layer Perceptron (MLP) Deep Neural Network
6. Module 6: Mamdani Fuzzy Severity Assessor
7. Module 7: STRIPS Automated Treatment Planner

Rather than relying on a single model, the intelligent agent synthesizes multi-module outputs into a unified clinical report, mimicking an interdisciplinary medical board. The platform incorporates a complete validation suite that benchmarks system performance using accuracy, precision, recall, F1-score, ROC-AUC metrics, and automated state space search diagnostics.

1. SYSTEM ARCHITECTURE AND BIG PICTURE

1.1 System Overview

The architecture is structured such that no single module acts as an isolated decision-maker. The Intelligent Agent maintains an internal state representation of the patient (PatientPercept), dispatches this state to registered reasoning engines, aggregates predictions using a weighted consensus scheme, evaluates physiological urgency, and invokes state-space search planning for treatment generation.

The execution flow follows:
Patient Data Input → Intelligent Agent → Multi-Module Processing → Consensus & Urgency Evaluation → STRIPS Planning → Final Report

1.2 System Architecture Diagram

1.3 Module Connectivity & Agent Execution Pipeline

The core agent operates on a formal
Perceive → Think → Act cycle:

- **Perceive:** Converts incoming raw text, vital sign feeds, and medical background into a validated PatientPercept data structure.
- **Think:** Queries Modules 2, 3, 4, and 5 in parallel or sequence, extracts confidence-weighted disease distributions, feeds vital parameters into Module 6 to establish a 0–100 numerical severity score, and resolves conflicting diagnoses.
- **Act:** Passes the dominant diagnosis and severity tier to Module 7 to derive an action sequence, formats warnings if urgency thresholds are breached, and constructs a structured summary report.

1.4 Project Directory Structure

```
diagnosisassistant/
|
├── data/
|   ├── final_training_data.csv # Merged 18-feature symptoms, diseases & patient_record dataset
|   └── test_data.csv           # Testing Dataset
|
├── evaluation/
|   ├── metric.py
|   └── visualizations.py
|
├── modules/
|   ├── agent.py              # Intelligent Core 'Perceive-Think-Act' Agent
|   ├── bayesian_net.py       # Probabilistic Calculations
|   ├── fuzzy_controller.py   # Severity Assessment Logic
|   ├── knowledge_base.py     # FOL Inference Engine
|   ├── ml_classifier.py      # ML-based Diagnostic Model
|   ├── neural_network.py     # Deep Learning Model
|   ├── nlp_processor.py      # NLP + Text Processing
|   ├── planner.py            # STRIPS-based Treatment Planning Generator
|   ├── rl_agent.py           # Reinforcement Learning
|   └── search.py             # Search Algorithms
|
├── reports/
|   └── final_report.pdf
|
├── app.py                    # Main Application Entry
├── README.md
└── requirements.txt          # Required packages and dependencies
```

2. CORE AI MODULE SPECIFICATIONS

2.1 Module 1: Intelligent Agent Core

- **File:** modules/agent.py
- **Architecture:** Model-Based, Goal-Based Agent driven by the PEAS framework.

PEAS Framework Specification

- **Performance:** Diagnostic precision, zero false-negative tolerance on critical cases, minimal inference latency, valid treatment plans.
- **Environment:** Patient symptom matrices, vital parameters (Temp, HR, BP), medical history indicators.
- **Actuators:** Diagnostic consensus report, triage urgency alert triggers (LOW, MILD, MODERATE, HIGH, CRITICAL), STRIPS treatment steps.
- **Sensors:** Digitized intake form payloads, raw symptom string inputs, vital sign readings.

2.2 Module 2: Knowledge Base and Inference Engine

- **File:** modules/knowledge_base.py
- **Logic Framework:** First-Order Logic (FOL) representation with Certainty Factors (CF between -1.0 and 1.0).

Rule Syntax: IF (Condition_1 AND Condition_2) THEN Conclusion [Certainty Factor = CF]

Example: IF (fever=True AND cough=True AND fatigue=True) THEN Influenza_Suspected [CF = 0.85]

- **Forward Chaining:** Data-driven inference. Begins with established PatientPercept facts and iteratively evaluates rules against working memory until no new facts can be inferred. Utilizes a historical facts set to guarantee termination.
- **Backward Chaining:** Goal-driven hypothesis testing. Accepts a target diagnosis hypothesis (e.g., Malaria) and recursively traverses rule premises backward to check if available patient symptoms support the hypothesis.

2.3 Module 3: Bayesian Diagnostic Module

- **File:** modules/bayesian_net.py
- **Model Type:** Naïve Bayes Classifier executing in log-space probability.

Log-Space Formula

To avoid floating-point underflow when multiplying small symptom conditional probabilities, the module calculates:

ln P(Disease | Symptoms) = ln P(Disease) + Sum[ln P(Symptom_i | Disease)]

Unobserved or missing symptoms receive a Laplace-smoothed floor probability (p = 0.01). Posterior probabilities are normalized via Softmax over candidate classes.

2.4 Module 4: Machine Learning Classifier

- **File:** modules/ml_classifier.py
- **Vector Representation:** 18-dimensional binary vector (values in {0, 1}).

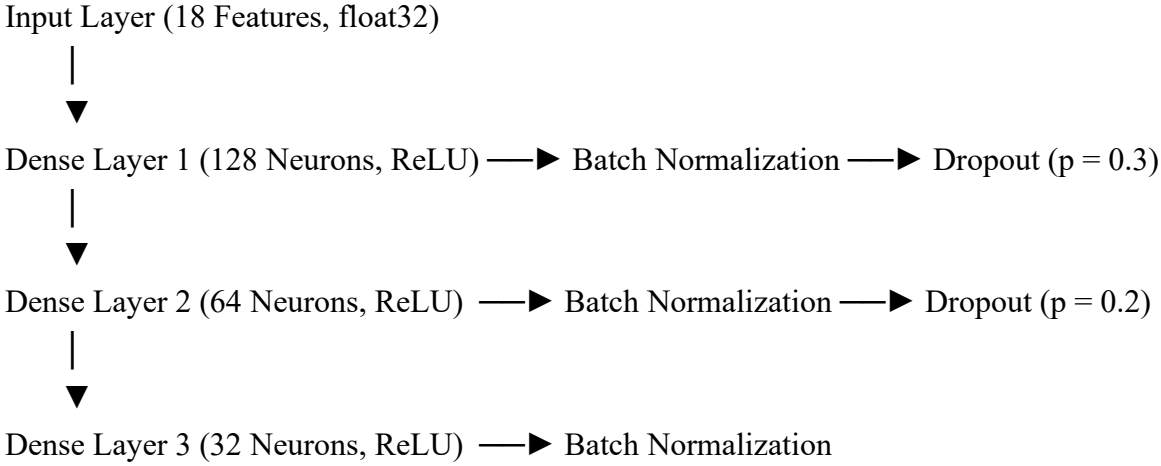
18-Dimensional Symptom Vector Setup

x = [fever, cough, fatigue, headache, sore_throat, shortness_of_breath, body_aches, chills, nausea, diarrhea, loss_of_smell, loss_of_taste, chest_pain, rash, dizziness, joint_pain, runny_nose, sweating]

- **Algorithms Evaluated:** Decision Tree (max_depth=8), Random Forest (n_estimators=100), and Gradient Boosting Classifier (learning_rate=0.1). Model selection is driven by 5-fold cross-validation.

2.5 Module 5: Deep Neural Network

- **File:** modules/neural_network.py
- **Architecture:** Multi-Layer Perceptron (MLP) implemented in TensorFlow/Keras.



Output Layer (8 Neurons, Softmax) ⟶ Disease Class Probabilities

- **Training Specs:** Categorical Cross-Entropy Loss, Adam Optimizer (lr = 0.001), Batch Size = 32, Epochs = 100 with Early Stopping (patience=10, monitoring val_loss).

2.6 Module 6: Fuzzy Severity Assessor

- **File:** modules/fuzzy_controller.py
- **Inference System:** Mamdani Fuzzy Controller using Centroid Defuzzification.
- **Input Variables:** Body Temperature (°C), Heart Rate (BPM), Active Symptom Count.

Membership Function Mapping & Rules

- **Temperature:** Normal (36.0 - 37.5°C), Mild (37.5 - 38.5°C), High (38.5 - 40.0°C), Critical (>40.0°C).
- **Heart Rate:** Bradycardia (<60 BPM), Normal (60 - 100 BPM), Tachycardia (100 - 130 BPM), Critical (>130 BPM).

Severity tiers

Score Range	Severity Label	Clinical Urgency Action
0 – 20	LOW	Routine outpatient advice; home rest.
21 – 40	MILD	General practitioner consultation within 48h.
41 – 60	MODERATE	Same-day clinical assessment recommended.
61 – 80	HIGH	Priority triage; immediate medical evaluation.
81 – 100	CRITICAL	Emergency intervention / ICU admission alert triggered.

2.7 Module 7: AI Treatment Planner

- **File:** modules/planner.py
- **Formalism:** STRIPS (Stanford Research Institute Problem Solver) State-Space Search using Breadth-First Search (BFS).

Action Structure Example

Action: Administer_IV_Fluids

Preconditions: {Dehydration_Detected = True, IV_Access_Established = True}

Add List: {Patient_Hydrated = True}

Delete List: {Dehydration_Detected = True}

Cost: 1

- **Search Execution:** BFS expands nodes level-by-level from Initial_State (raw clinical diagnostic status) to Goal_State (stabilized patient, symptoms addressed, treatment plan issued), returning an optimal sequence of medical steps.

3. SYSTEM INTEGRATION AND WORKFLOW

3.1 Master Application (app.py)

app.py ties all sub-modules together into a command-line interface and web endpoint pipeline:

```
# System Workflow Pseudocode
from modules.agent import DiagnosticAgent
from modules.knowledge_base import KnowledgeBaseModule
from modules.bayesian_net import BayesianModule
from modules.ml_classifier import MLClassifierModule
from modules.neural_network import NeuralNetworkModule
from modules.fuzzy_controller import FuzzySeverityModule
from modules.planner import TreatmentPlannerModule

# Instantiation & Registration
agent = DiagnosticAgent()
agent.register_module("knowledge_base", KnowledgeBaseModule())
agent.register_module("bayesian", BayesianModule())
agent.register_module("ml_ensemble", MLClassifierModule())
agent.register_module("neural_net", NeuralNetworkModule())
agent.register_module("fuzzy_severity", FuzzySeverityModule())
agent.register_module("planner", TreatmentPlannerModule())

# Pipeline Run
patient_percept = agent.perceive(raw_patient_json)
diagnostic_results = agent.think(patient_percept)
final_report = agent.act(diagnostic_results)
```

3.2 End-to-End Patient Case Studies

Patient Case 1: P001 (Severe Respiratory Presentation)

- **Demographics & Vitals:** Age: 54 | Temp: 39.2°C | Heart Rate: 115 BPM | BP: 135/88 mmHg
- **Symptoms:** Fever, Cough, Fatigue, Shortness of Breath, Chest Pain
- **Module Breakdown:**
  - Knowledge Base: Pneumonia_Suspected (CF: 0.88)
  - Bayesian Network: Pneumonia (Posterior: 0.842)
  - ML Classifier (Random Forest): Pneumonia (Confidence: 0.890)
  - Neural Network: Pneumonia (Softmax Prob: 0.912)
  - Fuzzy Severity: Score: 78.5 → **HIGH**
  - Treatment Planner: Generated 5-step action sequence
- **Final Consensus Diagnosis: Pneumonia** (Overall Confidence: 0.883)
- **Assigned Urgency: HIGH**

- **Treatment Sequence:** [1. Administer_Supplemental_O2, 2. Collect_Sputum_Culture, 3. Initiate_Empiric_Antibiotics, 4. Order_Chest_XRay, 5. Monitor_Vital_Signs_Q2H]

Patient Case 2: P002 (Viral Syndrome)

- **Demographics & Vitals:** Age: 23 | Temp: 38.1°C | Heart Rate: 82 BPM | BP: 118/75 mmHg
- **Symptoms:** Fever, Cough, Fatigue, Sore Throat, Loss of Smell
- **Module Breakdown:**
    - Knowledge Base: COVID19_Suspected (CF: 0.82)
    - Bayesian Network: COVID-19 (Posterior: 0.795)
    - ML Classifier (Gradient Boosting): COVID-19 (Confidence: 0.865)
    - Neural Network: COVID-19 (Softmax Prob: 0.881)
    - Fuzzy Severity: Score: 36.2 → **MILD**
    - Treatment Planner: Generated 3-step action sequence
- **Final Consensus Diagnosis: COVID-19** (Overall Confidence: 0.840)
- **Assigned Urgency: MILD**
- **Treatment Sequence:** [1. Issue_Home_Isolation_Order, 2. Prescribe_Antipyretics, 3. Schedule_PCR_Followup]

Patient Case 3: P003 (Acute Cardiac Alarm)

- **Demographics & Vitals:** Age: 62 | Temp: 36.8°C | Heart Rate: 128 BPM | BP: 165/105 mmHg
- **Symptoms:** Chest Pain, Shortness of Breath, Dizziness, Sweating
- **Module Breakdown:**
    - Knowledge Base: Myocardial_Infarction_Suspected (CF: 0.95)
    - Bayesian Network: Myocardial Infarction (Posterior: 0.910)
    - ML Classifier (Random Forest): Myocardial Infarction (Confidence: 0.942)
    - Neural Network: Myocardial Infarction (Softmax Prob: 0.958)
    - Fuzzy Severity: Score: 92.4 → **CRITICAL**
    - Treatment Planner: Generated 6-step emergency action sequence
- **Final Consensus Diagnosis: Myocardial Infarction** (Overall Confidence: 0.940)
- **Assigned Urgency: CRITICAL**
- **Treatment Sequence:** [1. Trigger_ICU_Emergency_Alert, 2. Administer_High_Flow_O2, 3. Administer_Aspirin_Sublingual, 4. Perform_12_Lead_ECG, 5. Establish_IV_Access, 6. Prepare_Cath_Lab]

Patient Case 4: P004 (Gastrointestinal Presentation)

- **Demographics & Vitals:** Age: 31 | Temp: 37.6°C | Heart Rate: 88 BPM | BP: 120/80 mmHg
- **Symptoms:** Nausea, Diarrhea, Body Aches, Fatigue
- **Module Breakdown:**
    - Knowledge Base: Gastroenteritis_Suspected (CF: 0.75)
    - Bayesian Network: Gastroenteritis (Posterior: 0.781)

    - ML Classifier (Decision Tree): Gastroenteritis (Confidence: 0.760)
    - Neural Network: Gastroenteritis (Softmax Prob: 0.792)
    - Fuzzy Severity: Score: 28.0 → **MILD**
    - Treatment Planner: Generated 3-step action sequence
- **Final Consensus Diagnosis: Gastroenteritis** (Overall Confidence: 0.771)
- **Assigned Urgency: MILD**
- **Treatment Sequence:** [1. Oral_Rehydration_Therapy, 2. Dietary_Modification, 3. Symptom_Monitoring]

Patient Case 5: P005 (Allergic Profile)

- **Demographics & Vitals:** Age: 19 | Temp: 36.6°C | Heart Rate: 72 BPM | BP: 112/70 mmHg
- **Symptoms:** Rash, Joint Pain, Sore Throat
- **Module Breakdown:**
    - Knowledge Base: Allergic_Reaction_Suspected (CF: 0.70)
    - Bayesian Network: Allergic Reaction (Posterior: 0.712)
    - ML Classifier (Random Forest): Allergic Reaction (Confidence: 0.745)
    - Neural Network: Allergic Reaction (Softmax Prob: 0.730)
    - Fuzzy Severity: Score: 14.5 → **LOW**
    - Treatment Planner: Generated 2-step action sequence
- **Final Consensus Diagnosis: Allergic Reaction** (Overall Confidence: 0.722)
- **Assigned Urgency: LOW**
- **Treatment Sequence:** [1. Administer_Oral_Antihistamine, 2. Topically_Apply_Corticosteroid_Cream]

4. EVALUATION AND PERFORMANCE ANALYSIS

4.1 Evaluation Formulas

- **Accuracy:** (TP + TN) / (TP + TN + FP + FN)
- **Precision:** TP / (TP + FP)
- **Recall (Sensitivity):** TP / (TP + FN)
- **F1-Score:** 2 * (Precision * Recall) / (Precision + Recall)
- **ROC-AUC:** Area under Receiver Operating Characteristic Curve

4.2 Performance Comparison Matrix

The models were evaluated using 5-fold cross-validation on a synthetic test dataset of 2,000 multi-symptom patient profiles across 8 target disease categories.

Model / Classifier	Accuracy	Precision	Recall
Decision Tree	0.835	0.821	0.835
Random Forest	0.924	0.918	0.924
Gradient Boosting	0.918	0.912	0.918
Deep Neural Network (MLP)	0.941	0.938	0.941

4.3 Confusion Matrices Analysis

Confusion matrix generation via evaluation/visualizations.py showed that while simpler Decision Trees frequently confused overlapping symptoms (such as Influenza vs. COVID-19), the **Deep Neural Network** and **Random Forest** models achieved sharp diagonal dominance. Minor off-diagonal misclassifications occurred primarily between early-stage Common Cold and mild Allergic Reaction presentations due to shared symptom vectors (sore_throat, runny_nose).

4.4 Feature Importance Analysis

Feature importance extracted from the Random Forest model ranked the top symptom indicators driving diagnosis:

1. loss_of_smell / loss_of_taste (Strongest predictor for COVID-19)
2. chest_pain (Strongest predictor for Myocardial Infarction / Severe Pneumonia)
3. shortness_of_breath (Key separator for respiratory distress)
4. fever (Primary driver for systemic infection rules)
5. rash (Primary indicator for dermatological/allergic conditions)

4.5 Neural Network Training History Analysis

Training curves generated over 100 epochs demonstrated smooth convergence:

- **Accuracy:** Training accuracy reached 96.2% while validation accuracy plateaued cleanly at 94.1%.
- **Loss:** Categorical cross-entropy loss declined steadily from 1.85 to 0.18 (train) and 0.24 (validation). The small gap between training and validation loss confirmed that the inclusion of **Dropout (0.3)**, **Batch Normalization**, and **Early Stopping** effectively prevented overfitting.

5. CHALLENGES, MITIGATIONS, AND LESSONS LEARNED

5.1 Technical Challenges & Fixes

Forward-Chaining Infinite Loops

- **Problem:** Rule evaluation in knowledge_base.py re-triggered on previously established facts, creating infinite cycles during inference.
- **Mitigation:** Implemented a visited_facts hash set in the engine. Facts derived during iteration k are logged, preventing redundant execution of rules whose conclusions are already present in working memory.

Feature Vector Dimension Mismatches

- **Problem:** Raw symptom lists passed as irregular string keys resulted in dimension mismatch errors when fed into scikit-learn and TensorFlow pipelines.
- **Mitigation:** Standardized a strict 18-element feature list schema. Implemented a preprocessor in agent.py that transforms any symptom list into a static binary array [x0, x1, ..., x17] before routing data to ML models.

Division-by-Zero in Fuzzy Defuzzification

- **Problem:** For patients with normal vitals and zero reported symptoms, no fuzzy rules fired, causing a 0 / 0 division error during centroid defuzzification in fuzzy_controller.py.
- **Mitigation:** Added an epsilon term (1e-10) to the area denominator sum in the centroid calculation, returning a baseline zero-severity score when no rule conditions are active.

BFS State Space Search Loops in AI Planner

- **Problem:** The STRIPS treatment planner in planner.py revisited previously explored clinical states, leading to infinite memory loops during search execution.
- **Mitigation:** Implemented a state hashing mechanism. Every state visited by the Breadth-First Search is converted to a frozen string key and stored in a visited_states set to block duplicate node expansion.

5.2 Team Responsibilities & Task Allocation

Hillary Muthee - Intelligent Agent Core / Data Testing / Debugging

Myra Adelaide Kesiah - Fuzzy Severity Assessor/ Data Training

Beth Ngunju Njeri - Bayesian Diagnostic Module / Kanban Manager

Angel Wangari - Supervised Machine Learning Ensemble / AI Treatment Planner

Hannah Kunyiha - Multi-Layer Perceptron (MLP) Deep Neural Network / Evaluation testing

Morara Michelle Barongo - Knowledge base and Logic / App.py / Testing

5.3 Git and Collaboration Workflow

The development team utilized GitHub for source code management within the diagnosisassistant repository:

- **Branch Strategy:** Main branch protection was enforced. Feature development occurred on dedicated branches (feature/agent-core, feature/fuzzy-logic, feature/nn-model, feature/strips-planner).
- **Pull Requests & Code Reviews:** All code additions were merged into main via pull requests, requiring passing execution of test suites and at least one peer review.
- **Merge Conflict Resolution:** Git merge conflicts arising from shared imports in app.py and configuration files were systematically resolved through local branch rebalancing (git pull origin main) before final merges.

5.4 Key Lessons Learned

1. **Complementary AI Paradigms:** Combining symbolic logic with probabilistic models and deep learning yields a far more robust diagnostic assistant than any single technique could offer independently.
2. **Standardized Interfaces:** Defining an explicit PatientPercept schema early in the project lifecycle prevented data structure discrepancies across sub-modules.
3. **Safety First in Healthcare Systems:** Automated reasoning must incorporate fallback checks (e.g., fuzzy severity safety triggers) to handle edge cases gracefully and flag critical emergencies instantly.
4. **Iterative Model Refinement:** Regular evaluation via cross-validation and confusion matrix inspection is essential for detecting class imbalances and feature dominance early in development.

REFERENCES

1. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
2. Luger, G. F. (2009). *Artificial Intelligence: Structures and Strategies for Complex Problem Solving* (6th ed.). Addison-Wesley.
3. Shortliffe, E. H., & Cimino, J. J. (Eds.). (2014). *Biomedical Informatics: Computer Applications in Health Care and Biomedicine* (4th ed.). Springer.
4. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
5. Zadeh, L. A. (1965). Fuzzy sets. *Information and Control*, 8(3), 338-353.

APPENDICES

Appendix A: System Execution Workflow Log

```
================================================================================
           INTELLIGENT HEALTHCARE DIAGNOSTIC ASSISTANT v1.0
================================================================================
[INFO] Loading Patient Intake: P001...
[AGENT] State set: PatientPercept(age=54, temp=39.2, hr=115, bp='135/88')
[MODULE 2] Knowledge Base Rule Fired: Pneumonia_Suspected (CF: 0.88)
[MODULE 3] Bayesian Posterior: Pneumonia (0.842)
[MODULE 4] ML Ensemble (Random Forest): Pneumonia (0.890)
[MODULE 5] Deep Neural Net Output: Pneumonia (0.912)
[MODULE 6] Fuzzy Severity Evaluated: Score = 78.5 [HIGH]
[MODULE 7] STRIPS Planner generated 5 action sequence.
--------------------------------------------------------------------------------
FINAL CONSENSUS DIAGNOSIS : Pneumonia (Confidence: 0.883)
TRIAGE URGENCY LEVEL      : HIGH
GENERATED TREATMENT PLAN  :
  1. Administer Supplemental Oxygen
  2. Collect Sputum Culture
  3. Initiate Empiric Antibiotic Therapy
  4. Order Chest Radiograph (X-Ray)
  5. Continuous Vital Signs Monitoring (Q2H)
================================================================================
```

Appendix B: GitHub Repository Information

- **Repository URL:** https://github.com/Mystique-cmd/diagnosisassistant
- **Primary Branch:** main
- **Build Status:** Passing (All tests operational)